Embedded Data Structure
(Lecture Note 15)

Sorting Algorithms (2)

2025. 5. 1

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

Review of the Last Class (1/2)

KU KONKUK UNIVERSITY

- **Handling a constructed data structure**
  - **Sorting algorithms (ascending or descending)**
    - **Two main operations : decision(compare) and exchange**
    - **Selection sort and bubble sort**

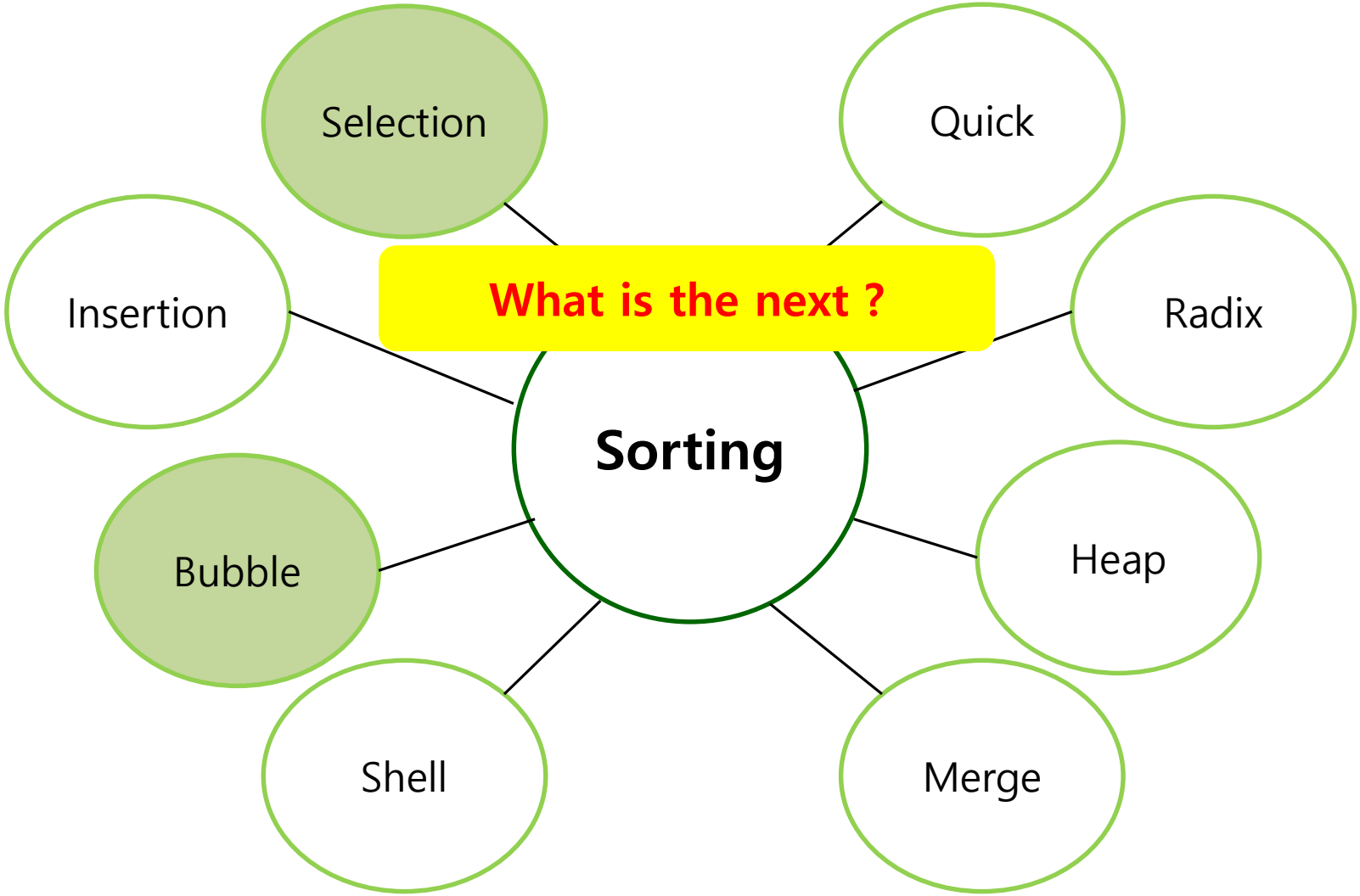
      ※ Remember scheme for each sorting algorithm

      ※ Remember the processing time for various cases
  - **Generalized version of sorting algorithms**
    - **Key point : using a void pointer**

      ※ Remember some rules for conversion (e.g., copy → memcpy)

Review of the Last Class (2/2)

- **Diversity of sorting algorithms**



Insertion Sort (1/6)

▪ Basic concept of insertion sorting

- **Insert a new key into sorted part as follows :**

Pseudo code :

```
1)  i = 1;
2)  j = i; t = a[i];
3)  while a[j-1] > t && j > 0,
      1)  a[j] = a[j-1];
      2)  j--;
4)  a[j] = t;
5)  i++ and go to the step 2)
```

For example,

int a[] = { 9, 4, 5, 1};

Draw your procedure !

9^1 4 5 1 3 9^2 9^3

4 9^1 5 1 3 9^2 9^3

4 5 9^1 1 3 9^2 9^3

1 4 5 9^1 3 9^2 9^3

1 3 4 5 9^1 9^2 9^3

1 3 4 5 9^1 9^2 9^3

Is this sorting stable ?

Insertion Sort (2/6)

▪ C Implementation

- **Using the previous pseudo code directly**

```
void insert_sort(int *a, int N)
{
    int i, j, t;
    for(i = 1; i<N; i++)
    {
        t = a[i];
        j = i;
        while(j > 0 && a[j-1] > t)
        {
            a[j] = a[j-1];
            j--;
        }
        a[j] = t;
    }
}
```

Which part consumes the processing time ?

Which part guarantees the stability ?

Insertion Sort (3/6)

KU KONKUK UNIVERSITY

- **Characteristics of insertion sort**
  - **It mostly performs fast compared to other algorithms !**
  - **It is useful for most of small records**

    ➔ **How about big-O notation ?**

※ Analysis of the processing time (using the tick)

N	Sorted	Random	Half-sorted	Reverse
10	0	0	0	0
100	0	0	0	1
500	0	10	2	20
1000	0	40	10	79
2000	0	160	48	315

What is the worst case ? Why ?

Insertion Sort (4/6)

▪ Optimization of insertion sort

- **Indirection sort : indexing !**

```
void indirect_insert_sort(int *a, int *index, int N)
{
    int i, j, t;
    for(i = 0; i<N; i++)
        index[i] = i;
    for(i = 1; i<N; i++)
    {
        t = index[i];
        j = i;
        while(j > 0 && [          ] )
        {
            index[j] = index[j-1];
            j--;
        }
        [          ]
    }
}
```

What does it mean ?

What does it mean ?

Insertion Sort (5/6)

- **Optimization of insertion sort – cont'd**
  - **Think about the procedure of "indirect" sorting**

Before sorting

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
A[i]	T	O	L	E	A	R	N	S	O	R	T	A	L	G	O	R	I	T	H	M
Index[i]	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

After sorting

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
A[i]	T	O	L	E	A	R	N	S	O	R	T	A	L	G	O	R	I	T	H	M
Index[i]	4	11	3	13	18	16	2	12	19	6	1	8	14	5	9	15	7	0	10	17

How can we access the left-most value ?

Insertion Sort (6/6)

- **Generalization of insertion sort**
  - **See the slide of the last class for your implementation**

```
void insert_sort(void *base, size_t nelem, size_t width,
                 int (*fcmp)(const void *, const void*))
{
    int i, j;
    void *t;

    Make your code !

}
```

Shell Sort (1/5)

KU KONKUK UNIVERSITY

▪ Basic concept of shell sort

- **Improvement of insertion sort using the h interval**
  - Shell sort is basically same with the insertion sort
  - For example, when h sufficiently decreases, it is very fast **Why ?**

➡ **Coarse-to-fine searching (sorting)**

Pseudo code :

1) Initialize h (step size)
2) If h < 1, then stop
3) i = 0
4) If i >= h, then go to the step 7)
5) Conduct insertion sort at (h + i)
6) i++ and go to the step 4)
7) Compute the next h (e.g., h = h/2) and go to the step 2)

Shell Sort (2/5)

- **Practical example**

T^1	O	L	E	A	R	N	S	O	R	T^2	A	L	G	O	R	I	T^3	H	M	h = 10, i = 0
T^1	O	L	E	A	R	N	S	O	R	T^2	A	L	G	O	R	I	T^3	H	M	i = 1
T^1	A	L	E	A	R	N	S	O	R	T^2	O	L	G	O	R	I	T^3	H	M	i = 2
T^1	A	L	E	A	R	N	S	O	R	T^2	O	L	G	O	R	I	T^3	H	M	i = 3
T^1	A	L	E	A	R	N	S	O	R	T^2	O	L	G	O	R	I	T^3	H	M	i = 4
T^1	A	L	E	A	R	N	S	O	R	T^2	O	L	G	O	R	I	T^3	H	M	i = 5
T^1	A	L	E	A	R	N	S	O	R	T^2	O	L	G	O	R	I	T^3	H	M	i = 6
T^1	A	L	E	A	R	I	S	O	R	T^2	O	L	G	O	R	N	T^3	H	M	i = 7
T^1	A	L	E	A	R	I	S	O	R	T^2	O	L	G	O	R	N	T^3	H	M	i = 8
T^1	A	L	E	A	R	I	S	H	R	T^2	O	L	G	O	R	N	T^3	O	M	i = 9
T^1	A	L	E	A	R	I	S	H	M	T^2	O	L	G	O	R	N	T^3	O	R	h = 5, i = 0
R	A	L	E	A	R	I	S	H	M	T^1	O	L	G	O	T^2	N	T^3	O	R	i = 1
R	A	L	E	A	R	I	L	H	M	T^1	N	S	G	O	T^2	O	T^3	O	R	i = 2
R	A	L	E	A	R	I	L	G	M	T^1	N	S	H	O	T^2	O	T^3	O	R	i = 3
R	A	L	E	A	R	I	L	G	M	T^1	N	S	H	O	T^2	O	T^3	O	R	i = 4
R	A	L	E	A	R	I	L	G	M	T^1	N	S	H	O	T^2	O	T^3	O	R	h = 2, i = 0
A	A	G	E	I	R	L	L	O	M	O	N	O	H	R	T^2	S	T^3	T^1	R	i = 1
A	A	G	E	I	H	L	L	O	M	O	N	O	R	R	R	S	T^2	T^1	T^3	h = 1, i = 0
A	A	E	G	H	I	L	L	M	N	O	O	O	R	R	R	S	T^2	T^1	T^3	

Shell Sort (3/5)

▪ C Implementation

- **Improvement of insertion sort using the h interval**

```
void shell_sort(int a*, int N)
{
    int i, j, k, h, v;
    for(h = N/2; h>0; h /= 2){
        for(i = 0; i<h; i++){
            for(j = i+h; j<N; j += h){
                v = a[j];
                k = j;
                while(k > h-1 && a[k-h] > v){
                    a[k] = a[k-h];
                    k -= h;
                }
                a[k] = v;
            }
        }
    }
}
```

Setting the interval

Compare with the interval

Think about h = 1

What does it mean ?

Shell Sort (4/5)

- **Characteristics of shell sort**
  - **It is very fast regardless of the size of records**
  - **It does not require the additional memory ! (cf. quick sort)**

➔ **How about big-O notation ? $O(N(\log N)^2)$**

※ Analysis of the processing time (using the tick)

N	Sorted	Random	Half-sorted	Reverse
100	0	0	0	1
500	1	2	2	2
1000	2	4	4	4
2000	6	9	8	7
5000	16	30	26	22
7000	22	42	38	31
10000	35	63	63	48

Shell Sort (5/5)

- **Improved version of shell sort**
  - **Interval : $h_n = 3*h_{n+1} + 1$, $h_1 = 1$**

Make your code !

※ Analysis of the processing time (using the tick)

N	Sorted	Random	Half-sorted	Reverse
100	0	0	0	0
500	1	1	1	1
1000	1	3	2	2
2000	3	8	7	5
5000	10	24	22	14
7000	15	34	32	21
10000	23	51	51	34

Summary

- **Sorting algorithms : data management**
  - **There is no optimal solution for all the cases**
  - **Remember the generalization scheme using void pointers**
  - **Today's topic : insertion / shell / distribution counting**